Framework de desarrollo seguro basado en métodos computacionales guiados por conocimiento

Informe del sprint 1

Walking skeleton y fase 4 inicial (Security Gate 1)

5 al 16 de octubre de 2026

Marcelo Acuña Gamboa — Participante 1: motor e IA

Abner Arboleda — Participante 2: plataforma y seguridad

Universidad de las Fuerzas Armadas – ESPE • Ingeniería de Software

Versión 1.0 • 16 de octubre de 2026

Índice

1. Resumen del sprint **2**
- 1.1. Objetivo e hito . . . 2
- 1.2. Indicadores . . . 2

2. Trabajo realizado **3**
- 2.1. Cumplimiento del cronograma . . . 3
- 2.2. Plataforma y seguridad . . . 3
  - 2.2.1. Servicio de referencia `usuarios` . . . 3
  - 2.2.2. Contenerización . . . 4
  - 2.2.3. Security Gate 1 . . . 4
  - 2.2.4. Reglas Semgrep propias (v1) . . . 4
- 2.3. Motor e IA . . . 5

3. Estado del pipeline al cierre **7**
- 3.1. Walking skeleton hasta el Gate 1 . . . 7
- 3.2. Fases del pipeline . . . 7
- 3.3. Correcciones del pipeline del 16 de octubre . . . 7

4. Incidencias y lecciones aprendidas **9**

5. Seguridad del propio framework **10**

6. Planificación del sprint 2 **11**
- 6.1. Objetivo e hito . . . 11
- 6.2. Dependencias y preparación . . . 11

A. Evidencias **13**

1. Resumen del sprint

1.1 Objetivo e hito

El sprint 1 corresponde al principio P3 del plan técnico, *primero de punta a punta, después inteligente*. Su objetivo era construir las herramientas del Security Gate 1 y hacer que un microservicio NestJS escrito a mano lo superara en GitHub Actions, con sus hallazgos registrados en Neo4j. También había que elegir el proveedor de LLM.

Hito de control del 16 de octubre de 2026

Fase 4 inicial: un microservicio escrito a mano pasa el Security Gate 1 en CI. Si no se cumple, no se avanza a la generación con LLM.

Resultado: hito cumplido

El servicio de referencia `usuarios` superó el Security Gate 1 en la ejecución 38955751601 de GitHub Actions sobre la rama `dev` (16/10/2026). Las ocho comprobaciones terminaron en *OK* al primer intento y los cinco archivos SARIF se publicaron en la pestaña Security. La fase 4 del motor registró la ejecución en Neo4j. Se completaron todas las tareas planificadas del sprint y se puede pasar al sprint 2.

1.2 Indicadores

Tabla 1.1: Indicadores del sprint 1 medidos en la última ejecución sobre `dev`.

Indicador	Valor
Días y horas del sprint	9 días, 72 horas de equipo (36 por integrante)
Tareas planificadas completadas	18 de 18 (9 por integrante)
Pull requests fusionados en `dev`	17 (#137 a #155), todos con revisión del otro integrante
Duración del pipeline completo	3 min 25 s (Security Gate 1: 46 s)
Resultado del Security Gate 1	8 de 8 comprobaciones en *OK*, 0 hallazgos
Reglas Semgrep aplicadas	82: 74 públicas (`p/typescript`, `p/nodejs`) y 8 propias
Casos de prueba de las reglas propias	23 detecciones esperadas, 23 obtenidas; 0 falsos positivos en los casos *ok*
Pruebas del motor (pytest)	106 pruebas en verde, 95 % de cobertura de `orchestrator/`
Pruebas del servicio (Jest)	16 pruebas en verde, 80,8 % de cobertura de sentencias
Esquema de Neo4j	14 restricciones y 7 índices, aplicados en cada ejecución de CI
Proveedor de LLM	Groq (`openai/gpt-oss-20b` y `gpt-oss-120b`)
Alertas abiertas en GitHub Security	0

2. Trabajo realizado

2.1 Cumplimiento del cronograma

La tabla compara el cronograma diario del plan con lo entregado. La columna PR indica el pull request con el que cada tarea entró en `dev`. Las filas de un solo bloque son trabajo conjunto.

Tabla 2.1: Tareas del sprint 1 por integrante y pull request asociado.

Fecha	Marcelo (motor e IA)	Abner (plataforma y seguridad)	PR Marcelo	PR Abner
Sprint 1: Walking skeleton y fase 4 inicial (Security Gate 1)				
05/10 lun	Esqueleto del motor: uv, Typer, FastAPI y configuración.	`docker-compose.dev.yml` con Neo4j, RabbitMQ y Keycloak.	#139	#137
06/10 mar	Interfaz `ProveedorLLM` y adaptadores (Claude, Gemini, Ollama).	Servicio de referencia `usuarios`: módulo, DTO con class-validator y Swagger.	#140	#138
07/10 mié	Prueba comparativa con 5 requisitos; elección de Groq.	Prisma + PostgreSQL y pruebas Jest base.	#141	#142
08/10 jue	Modelo común de hallazgos y lector SARIF.	Dockerfile multietapa con Hadolint; workflow con un paso por fase.	#144	#143
12/10 lun	Esquema de Neo4j: restricciones e índices en `graph/`.	Gate 1 en CI: tsc, ESLint, Semgrep, Gitleaks, Trivy y Hadolint.	#148	#146
13/10 mar	Registro de ejecuciones y hallazgos como nodos en Neo4j.	SARIF en la pestaña Security; `npm ci --ignore-scripts` y `.npmrc`.	#151	#147
14/10 mié	Comando `framework run --fase 4` integrado al workflow.	Reglas Semgrep propias v1 para NestJS y Prisma.	#152, #153	#149
15/10 jue	La fase 4 lee los SARIF del Gate 1 y los registra en Neo4j.	Gate 1 estable en el runner: reintentos, resumen y caché de Trivy.	#155	#150
16/10 vie	Pruebas del motor con pytest y guía de demostración.	Correcciones del pipeline, este informe y planificación del sprint 2.	#155	este PR

Algunas tareas se fusionaron uno o dos días después de su fecha planificada; por ejemplo, #143 se fusionó el 11/10 y #151 el 16/10. Estos retrasos se absorbieron dentro del sprint y no afectaron al hito.

2.2 Plataforma y seguridad

2.2.1 Servicio de referencia `usuarios`

Es un microservicio NestJS escrito a mano que sirve de banco de pruebas para todas las fases antes de que exista la generación con LLM. Expone tres operaciones REST (crear, listar y consultar por identificador) documentadas con Swagger. Persiste los datos en PostgreSQL

mediante Prisma y valida la entrada con un `ValidationPipe` global en modo `whitelist`. Las consultas de lista tienen un límite de 100 registros. Las 16 pruebas Jest cubren el DTO, el controlador y el servicio.

2.2.2 Contenerización

El Dockerfile tiene tres etapas: compilación, dependencias de producción e imagen final. La imagen final contiene solo el JavaScript compilado y las dependencias de producción, se ejecuta con un usuario sin privilegios (`USER 1000:1000`) e incluye `HEALTHCHECK`. Hadolint no reporta ningún aviso, ni siquiera de nivel *info*.

2.2.3 Security Gate 1

El Gate 1 se ejecuta en la fase 4 del workflow mediante `scripts/gate1.sh`. Aplica ocho comprobaciones al servicio de referencia y genera un SARIF por herramienta.

Tabla 2.2: Comprobaciones del Security Gate 1 y resultado en la ejecución del hito.

Herramienta	**Qué busca**	**SARIF**	**Resultado**
`tsc --noEmit`	Compilación y tipos	–	OK
ESLint + security	Patrones inseguros de JavaScript y estilo	–	OK
Semgrep	SAST con 74 reglas públicas y 8 propias	`semgrep`	OK
`semgrep --test`	Que las reglas propias detecten sus casos de prueba	–	OK
Gitleaks	Secretos en todo el historial de Git	`gitleaks`	OK
Trivy fs	Vulnerabilidades HIGH y CRITICAL en `package-lock.json` y `uv.lock`	`trivy-fs`	OK
Trivy config	Configuraciones inseguras en el Dockerfile	`trivy-config`	OK
Hadolint	Buenas prácticas del Dockerfile	`hadolint`	OK

El script distingue entre *hallazgos*, cuando la herramienta funciona y encuentra problemas, y *errores de la herramienta*, como fallos de red o de descarga. Los hallazgos ponen el Gate en rojo sin reintentar. Los errores se reintentan hasta dos veces, para que un fallo transitorio del runner no se confunda con un problema de seguridad. Al final, el script escribe una tabla de resultados en el resumen de la ejecución.

2.2.4 Reglas Semgrep propias (v1)

Las reglas de `semgrep_rules/` forman parte de la base de conocimiento (principio P5). Cada una declara el CWE y la categoría OWASP que detecta, para que el cargador del sprint 2 pueda enlazarlas en Neo4j. El archivo `nestjs-seguridad.ts` contiene los casos de prueba, y el pipeline comprueba en cada ejecución que todas las reglas siguen detectándolos.

Tabla 2.3: Reglas Semgrep propias, versión 1.

Regla	Qué detecta	Severidad	CWE
`nestjs-validationpipe-sin-whitelist`	Asignación masiva por propiedades no declaradas	WARNING	915
`prisma-sql-crudo-inseguro`	`$queryRawUnsafe` y `$executeRawUnsafe`	ERROR	89
`nestjs-cors-abierto`	CORS abierto a cualquier origen	WARNING	942
`jwt-secreto-en-codigo`	Secreto JWT escrito en el código	ERROR	798
`jwt-ignora-expiracion`	Tokens vencidos aceptados	ERROR	613
`tls-verificacion-desactivada`	Certificados TLS sin verificar	ERROR	295
`hash-criptografico-debil`	MD5 o SHA-1	WARNING	327
`prisma-findmany-sin-limite`	Consultas de lista sin `take`	WARNING	770

2.3 Motor e IA

- **Esqueleto del motor** (#139): proyecto Python con uv, CLI `framework` con Typer, API con FastAPI y configuración por variables de entorno.
- **Adaptador de LLM** (#140, #141): interfaz común con salida estructurada en Pydantic y adaptadores para Claude, Gemini, Ollama y Groq.
- **Modelo de hallazgos y lector SARIF** (#144): convierte la salida de cualquier herramienta al modelo común, como pide el principio P2. Se probó con ejemplos y con salidas reales de Hadolint y Gitleaks.
- **Neo4j** (#148, #151): 14 restricciones y 7 índices, y registro de nodos `Ejecucion` y `Hallazgo` enlazados con su `CWE`.
- **Fase 4 en CI** (#152, #153, #155): el workflow levanta un Neo4j propio de la ejecución, ejecuta las pruebas del motor, aplica el esquema y registra los hallazgos del Gate 1. Los SARIF de ejemplo sirven para mostrar cómo se ve el grafo cuando sí hay hallazgos.

Tabla 2.4: Comparación de proveedores de LLM con cinco requisitos (7/10/2026).

Criterio	Gemini	Groq
JSON válido al primer intento	5/5	5/5
Clasificación RF/RNF correcta	5/5	5/5
Cobertura de etiquetas de seguridad	0,80	0,77
TypeScript generado / que compila	1/5 / 1	5/5 / 5
Latencia media de generación	110,3 s	2,0 s
Reintentos por saturación o cuota	10	0

Decisión: Groq como proveedor de LLM

Las dos opciones extraen requisitos con la misma calidad. Groq, en cambio, generó y compiló todo el TypeScript sin reintentos y fue unas 55 veces más rápido en generación. Gemini agotó la cuota gratuita durante la prueba. El adaptador permite cambiar de proveedor sin tocar el resto del motor.

3. Estado del pipeline al cierre

3.1 Walking skeleton hasta el Gate 1

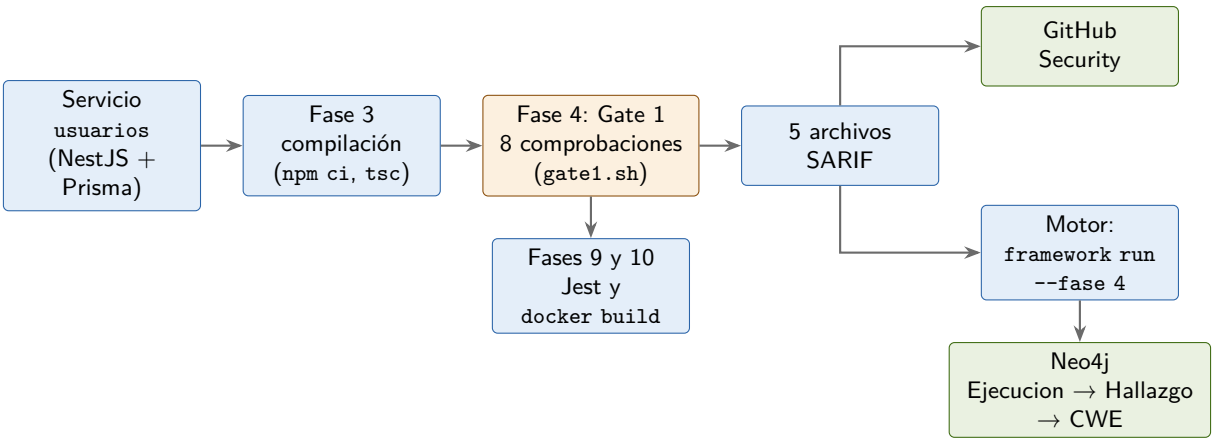


Figura 3.1: Recorrido del walking skeleton en cada ejecución de GitHub Actions.

3.2 Fases del pipeline

Tabla 3.1: Estado de cada fase del workflow al cierre del sprint 1.

Fase	Nombre	Estado	Detalle
1–2	Requisitos y arquitectura	Pendiente	Sprint 3.
3	Generación y compilación	Versión inicial	Compila el servicio de referencia (20 s).
4	Security Gate 1	Versión inicial	8 comprobaciones, SARIF en Security y registro en Neo4j (46 s).
5–6	Refinamiento iterativo	Pendiente	Sprint 5.
7	Integración	Pendiente	Keycloak en el sprint 2; Traefik y RabbitMQ en el sprint 5.
8	Security Gate 2	Pendiente	Spectral y Conftest el 21 y 22/10.
9	Pruebas automatizadas	Versión inicial	Jest con cobertura (3 s).
10	Contenerización	Versión inicial	`docker build` de la imagen (31 s).
11	DAST y Security Gate 3	Pendiente	Trivy image y Syft el 26/10; ZAP y Nuclei el 27/10.

3.3 Correcciones del pipeline del 16 de octubre

En la revisión de la última ejecución aparecieron dos avisos de GitHub y un problema de la caché. Se corrigen en el pull request de este informe.

Tabla 3.2: Correcciones aplicadas al workflow al cierre del sprint.

Problema	**Corrección**
`actions/cache@v4` y `actions/upload-artifact@v5` usan Node.js 20, que GitHub ya no admite.	Se actualizan a `cache@v5` y `upload-artifact@v6`, que usan Node.js 24. El único cambio entre versiones es el entorno de ejecución.
`ubuntu-latest` pasa a Ubuntu 26 desde el 19/10/2026.	Se fija `ubuntu-24.04`, para que el entorno no cambie a mitad del proyecto y los experimentos DSR sean reproducibles.
La base de Trivy (unos 130 MB) se guardaba como una entrada de caché nueva en cada ejecución.	La clave pasa a ser diaria: una sola entrada por día, sin agotar el límite de caché del repositorio.

4. Incidencias y lecciones aprendidas

Ninguna de las incidencias se resolvió relajando el Gate. Cada una se corrigió en su origen o, en el caso de los falsos positivos, se documentó con su justificación.

Tabla 4.1: Incidencias del sprint 1 y su corrección.

Incidencia	Causa	Corrección	PR
Gitleaks marca un secreto en el historial	Token de ejemplo del badge de CircleCI en la plantilla README de NestJS. No es una credencial real.	Se eliminó del README y se registraron sus huellas en `.gitleaksignore`, sin repetir el valor en ese archivo.	#146
Trivy fs con 2 vulnerabilidades HIGH	Dependencias transitivas del CLI de Prisma: `deepmerge-ts` (CVE-2026-40345) y `mysql2` (GHSA-3f6p-5ww8-9rcr).	`overrides` de npm a las versiones corregidas, tras verificar `prisma generate`, build, pruebas e imagen.	#146
Hadolint DL3066	`USER node` es un nombre; algunos sistemas no pueden comprobar que no sea administrador.	`USER 1000:1000`, el mismo usuario identificado por número.	#149
Scripts de instalación de npm	Un paquete comprometido puede ejecutar código al instalarse.	`--ignore-scripts` en CI y Dockerfile, y `ignore-scripts=true` en `.npmrc`.	#147
Fallos de herramienta confundidos con hallazgos	Un error de red o de descarga en el runner dejaba el Gate en rojo igual que un hallazgo.	`gate1.sh` separa ambos casos, reintenta los errores y precarga las imágenes.	#150
SARIF rechazados por GitHub	Las herramientas corren como root en contenedores y pueden dejar archivos vacíos.	El script devuelve la propiedad al usuario del runner y elimina los SARIF vacíos.	#150
Riesgo de `$'\r'` en bash	Git en Windows convierte los finales de línea a CRLF.	`.gitattributes` con `*.sh text eol=lf`.	#150
Workflow en rojo tras fusionar #152	`astral-sh/setup-uv@v10` no existe como etiqueta.	Versión fijada `v10.3.0`.	#153
Trivy se cuelga en local	En Windows recorre `node_modules` a través del montaje de Docker y supera los 5 minutos.	Solo ocurre en local. En el runner tarda segundos, así que se valida con el CI o con una copia limpia.	–

Lección principal

Casi todas las incidencias se debieron a diferencias entre el entorno local y el runner: Windows frente a Linux, finales de línea o versiones no fijadas. Probar cada cambio en un entorno equivalente al runner (un clon limpio, en Linux, con las mismas imágenes) permite detectarlas antes de abrir el PR. Fijar versiones de herramientas, acciones y sistema del runner es lo que hace reproducible el Gate.

5. Seguridad del propio framework

El capítulo 10 del plan exige que el framework se aplique a sí mismo sus principios. Estos son los controles que ya están activos al cierre del sprint 1:

Tabla 5.1: Controles del capítulo 10 implementados en el sprint 1.

Riesgo	**Control activo**
Scripts de instalación de npm	`npm ci --ignore-scripts` en CI y en las etapas del Dockerfile; `.npmrc` del servicio con `ignore-scripts=true`.
Secretos del framework	`.env` fuera de Git; claves en GitHub Secrets; Gitleaks revisa todo el historial en cada ejecución.
Dependencias del framework	Trivy fs revisa `package-lock.json` (servicio) y `uv.lock` (motor).
Manipulación del conocimiento	Toda la integración pasa por pull request con aprobación del otro integrante y CODEOWNERS; `semgrep --test` comprueba las reglas propias en cada ejecución.
Contenedores	Imagen final sin herramientas de compilación y con usuario no root.
Acceso a servicios locales	Los puertos del `docker-compose.dev.yml` se publican solo en `127.0.0.1`; el Neo4j de CI vive solo durante la ejecución.
Permisos del workflow	`contents: read` por defecto; `security-events: write` solo en el trabajo que publica SARIF.

6. Planificación del sprint 2

6.1 Objetivo e hito

Hito de control del 30 de octubre de 2026

Walking skeleton completo (tres gates) con avisos en Telegram. Es el primer hito importante del proyecto: no se empieza el sprint 3 sin haberlo cumplido.

El sprint 2 va del 19 al 30 de octubre: 10 días y 80 horas de equipo. Construye la base de conocimiento v1 con GraphRAG, las políticas OPA de los tres gates, las versiones iniciales de los Gates 2 y 3 sobre el servicio de referencia y los avisos de Telegram.

Tabla 6.1: Cronograma del sprint 2 por integrante.

Fecha	Marcelo (motor e IA)	Abner (plataforma y seguridad)
19/10 lun	Base de conocimiento: patrones de microservicios en YAML.	Notificador de Telegram: bot y mensaje de progreso editable.
20/10 mar	Controles OWASP ASVS y catálogo CWE / OWASP API Top 10.	Notificaciones en cada paso del workflow y aviso de fallo.
21/10 mié	Cargador YAML → Neo4j (nodos y relaciones).	Gate 2 inicial: Spectral con reglas OWASP API.
22/10 jue	Índice vectorial; fragmentos y embeddings de OWASP/CWE.	Conftest con políticas Rego para Docker Compose.
23/10 vie	Consulta GraphRAG (vectores + relaciones).	Realm de Keycloak y guard JWT en el servicio de referencia.
26/10 lun	Políticas Rego de los Gates 1 y 2 con `opa test`.	Gate 3 inicial: Trivy image y Syft (SBOM).
27/10 mar	Política Rego del Gate 3 con justificación.	ZAP API scan con límite de tiempo y Nuclei.
28/10 mié	Benchmark Neo4j frente a PostgreSQL + pgvector.	Conversor de resultados de ZAP a SARIF.
29/10 jue	Ambos: walking skeleton completo (Gates 1–3 con OPA en CI).	
30/10 vie	Ambos: revisión, demostración con avisos en Telegram, informe del sprint 2 y planificación del sprint 3.	

6.2 Dependencias y preparación

La revisión del sprint 1 deja varias dependencias entre tareas que conviene resolver al empezar el sprint:

- **Telegram:** los secretos `TELEGRAM_BOT_TOKEN` y `TELEGRAM_CHAT_ID` ya están en GitHub Secrets, así que el notificador puede empezar el 19/10 sin bloqueo.
- **Spectral (21/10)** necesita el documento OpenAPI del servicio. Habrá que exportar `openapi.json` en CI desde la configuración de Swagger, sin levantar la base de datos.
- **Guard JWT (23/10):** el código nuevo de autenticación quedará cubierto automáticamente por las reglas propias `jwt-secreto-en-codigo` y `jwt-ignora-expiracion` del Gate 1.

- **Trivy image y Syft (26/10)** reutilizan la imagen `fw-usuarios:ci` que ya construye la fase 10.
- **ZAP y Nuclei (27/10)** necesitan el servicio en marcha dentro del runner. Hay que añadir PostgreSQL como contenedor de servicio en el workflow y aplicar las migraciones (`prisma migrate deploy`) antes de arrancarlo.
- **Políticas OPA (26–29/10):** `gate1.sh` ya produce los SARIF y una tabla de resultados. La política `gate1.rego` debe tomar esa decisión, y el script pasará a limitarse a ejecutar las herramientas.
- **Duración del CI:** con DAST, el pipeline crecerá desde los 3,5 minutos actuales. El límite de tiempo de ZAP y las cachés de imágenes deben mantenerlo por debajo de los 30 minutos de `timeout` del trabajo.

A. Evidencias

Tabla A.1: Dónde comprobar cada resultado del sprint.

Evidencia	**Ubicación**
Repositorio	`https://github.com/PracticasESPE/Framework_Desarrollo_Seguro`
Ejecución del hito en `dev`	`https://github.com/PracticasESPE/Framework_Desarrollo_Seguro/actions/runs/38955751601`
Pestaña Security (SARIF)	`https://github.com/PracticasESPE/Framework_Desarrollo_Seguro/security/code-scanning`
Tablero del proyecto	`https://github.com/orgs/PracticasESPE/projects/1`
Guía de demostración	`docs/sprints/sprint1-demostracion.md`
Script del Gate 1	`scripts/gate1.sh`
Reglas Semgrep y casos de prueba	`semgrep_rules/`
Comparación de LLM	`evaluation/comparacion_llm/resultados.md`
Esquema de Neo4j	`graph/01_restricciones.cypher`, `graph/02_indices.cypher`

Cómo reproducir el Gate 1 en local

Hace falta Docker en marcha y una terminal Linux (o WSL); en Windows, Trivy se vuelve muy lento al leer el repositorio a través del montaje de Docker. Desde la raíz del repositorio:

```
cd services/usuarios && npm ci --ignore-scripts && npx prisma generate && cd ../..
bash scripts/gate1.sh
docker run --rm -v "$PWD:/src" semgrep/semgrep:1.172.0 semgrep --test semgrep_rules/
```